

PYTHON FOUNDATIONS SERIES | CHAPTER

Object-Oriented Programming

Classes, Objects & OOP Principles in Python 3

*Classes • Inheritance • Encapsulation • Abstraction • Polymorphism • Data Classes •
OOP for Data Science*

This chapter introduces **Object-Oriented Programming (OOP)** using Python 3 — the programming paradigm that underpins virtually every major library in the Data Science and AI ecosystem, from Pandas and Scikit-Learn to PyTorch and TensorFlow. Whether you are building a data pipeline, defining a machine learning model architecture, or designing a multi-agent AI system, OOP is the language of professional software engineering. This chapter builds the foundation from first principles.

What You Will Learn:

- What a class is and how it differs from an object (instance)
- How to design classes with attributes and methods for real-world entities
- Constructor methods (`__init__`, `__new__`) and dunder (double underscore) methods
- The four pillars of OOP: Abstraction, Encapsulation, Inheritance, and Polymorphism
- Class attributes vs. instance attributes — scope and use cases
- Advanced patterns: `__slots__` for memory efficiency, `@dataclass` for boilerplate reduction
- Why OOP is indispensable for Data Science: Pandas, Scikit-Learn, TensorFlow, and PyTorch
- Fully annotated Python notebook cells — from a blank class to a complete inheritance hierarchy

SECTION 01 | Theory & Concepts

Section 1: Theory — Object-Oriented Programming from First Principles

1.1 The Programming Paradigm Shift: Why OOP?

Before Object-Oriented Programming became the dominant paradigm, programs were written as long sequences of instructions — procedural programming. This works fine for small scripts, but as programs grow, procedural code becomes a tangled web of global variables, duplicated logic, and brittle dependencies that is extremely difficult to maintain, extend, or debug.

OOP reorganises code around a fundamentally different mental model: instead of thinking about what steps a program should execute, you think about what entities exist in your problem domain, what data they hold, and what actions they can perform. This shift produces code that is naturally modular, reusable, and aligned with how humans understand the world.

Python's Deep Commitment to OOP:

In Python, EVERYTHING is an object — integers, strings, lists, functions, modules, and even classes themselves are objects.

This means every Python value has attributes (data) and methods (behaviour) accessible via dot notation.

Example: `'42'.bit_length()` — even the integer 42 is an object with methods.

Understanding OOP is therefore not optional in Python — it is understanding how Python itself works at a fundamental level.

1.2 Classes and Objects: Blueprint vs. Instance

The class is the blueprint — a template that defines what data an entity will hold (attributes) and what actions it can perform (methods). The class itself is not a real thing in your program; it is a definition.

The object (also called an instance) is a real, concrete thing created from the blueprint. Every object gets its own private copy of the attributes defined by the class, and can independently change those values without affecting other objects of the same class.

Concept	Analogy & Python Reality
Class	An architectural blueprint for a house. The blueprint itself is not a house — you cannot live in it. In Python: <code>class Employee: ...</code> — a definition, not yet a real employee.
Object (Instance)	A house built from the blueprint. It has real walls, real rooms, a specific address. In Python: <code>employee1 = Employee('Ji-Soo', 38, 'developer', 1200)</code> — a real employee object in memory.
Attribute	A specific detail of a house: 'this house has 3 bedrooms.' In Python: <code>employee1.name = 'Ji-Soo'</code> — data stored on a specific object.

Method	An action a house can support: 'add a room.' In Python: <code>employee1.increase_salary(10)</code> — a function that operates on a specific object's data.
Instantiation	The act of constructing a house from the blueprint. In Python: calling <code>Employee(...)</code> triggers <code>__new__()</code> and <code>__init__()</code> to create and initialise a new object in memory.

1.3 Real-World Class Modelling

A well-designed class directly reflects a real-world entity in your problem domain. The attributes capture the entity's state; the methods capture its behaviour. Consider these two examples:

Entity	Attributes (State)	Methods (Behaviour)
Employee	name, age, position, salary	<code>increase_salary()</code> , <code>info()</code> , <code>get_department()</code>
Dog	color, age, breed, weight	<code>walking()</code> , <code>barking()</code> , <code>chase()</code> , <code>fetch()</code>
BankAccount	owner, balance, account_number, currency	<code>deposit()</code> , <code>withdraw()</code> , <code>get_balance()</code> , <code>transfer()</code>
DataPipeline	source_path, target_path, schema, status	<code>load()</code> , <code>transform()</code> , <code>validate()</code> , <code>save()</code>

Notice the pattern: attributes are **nouns** (name, salary, breed); methods are **verbs** (`increase_salary`, `barking`, `validate`). This noun/verb distinction is a reliable heuristic for class design — if you can naturally say 'an employee HAS a salary' (noun → attribute) or 'an employee CAN increase their salary' (verb → method), your design is on solid ground.

1.4 Instance Attributes and Dot Notation

When an object is created, Python allocates memory for it and populates its instance attributes — the per-object data values defined in the constructor. Instance attributes are accessed and modified using **dot notation**: `object.attribute_name`.

Two critically important properties of instance attributes:

- **Independence:** Each object has its own private copy. Changing `employee1.name` to 'Cynthia' does not affect `employee2.name`. Objects are completely independent in their data.
- **State representation:** The collection of an object's current attribute values is called its state. A `BankAccount` object with `balance=500` is in a different state than one with `balance=0`, even though both were created from the same class.

1.5 Instance Methods and the self Parameter

Methods are functions defined inside a class. The critical distinguishing feature is the **self** parameter — always the first parameter of every instance method. **self** is a reference to the specific object the method is being called on.

When you write `employee2.increase_salary(20)`, Python automatically translates this to `Employee.increase_salary(employee2, 20)` — it passes the object as the first argument. This is how methods know which object's data to operate on.

The self Convention:

self is a convention, not a keyword. You could technically name it anything, but you should ALWAYS name it **self**.

self gives methods access to the object's attributes: `self.salary`, `self.name`, `self.age`.

self allows methods to call other methods on the same object: `self.validate()` called inside `self.save()`.

Without **self**, a function defined inside a class would have no way to know which object it belongs to.

1.6 Constructor Methods: `__new__` and `__init__`

When you call `Employee('Ji-Soo', 38, 'developer', 1200)`, Python executes a two-phase object creation process:

Phase	Method & Responsibility
Phase 1: Creation	<code>__new__(cls, ...)</code> — called first. Allocates a new, empty object in memory. Returns the raw, uninitialised object. Rarely overridden directly — the default implementation from object is almost always sufficient.
Phase 2: Initialisation	<code>__init__(self, ...)</code> — called second, with the newly created object passed as <code>self</code> . Your job in <code>__init__</code> is to assign all initial values to the object's attributes. This is where you write <code>self.name = name</code> , <code>self.age = age</code> , etc.

After `__init__` completes, the object is fully initialised and ready to use. The key guarantee: **every object starts in a valid, predictable state** — you never have objects with missing or undefined attributes, as long as `__init__` is correctly written.

1.7 Dunder Methods: Python's Magic Interface

Dunder methods ("double underscore" methods, also called magic methods or special methods) are methods with a specific naming convention: `__method_name__`. Python automatically calls these methods in response to built-in operations — you almost never call them directly.

Dunder Method	When Python Calls It & What It Enables
<code>__init__(self, ...)</code>	Called when creating a new object. Sets up initial attribute values.
<code>__new__(cls, ...)</code>	Called before <code>__init__</code> . Allocates the object in memory. Used for singleton patterns, immutable types.
<code>__str__(self)</code>	Called by <code>print()</code> and <code>str()</code> . Should return a human-readable string description of the object. Example: 'Employee: Ji-Soo (developer, age 38)'.
<code>__repr__(self)</code>	Called by <code>repr()</code> and in the REPL. Should return a string that could recreate the object. Example: "Employee('Ji-Soo', 38, 'developer', 1200)".
<code>__len__(self)</code>	Called by <code>len()</code> . Allows your objects to respond to <code>len()</code> like lists do.
<code>__eq__(self, other)</code>	Called by <code>==</code> . Allows custom equality comparison between objects.
<code>__lt__(self, other)</code>	Called by <code><</code> . Enables sorting and comparison operators.
<code>__add__(self, other)</code>	Called by <code>+</code> . Enables operator overloading for custom addition.
<code>__getitem__(self, key)</code>	Called by <code>obj[key]</code> . Makes your objects subscriptable like lists and dicts.
<code>__enter__</code> / <code>__exit__</code>	Enable the <code>with</code> statement. Used for context managers (file handling, database connections).

1.8 The Four Pillars of OOP

Object-Oriented Programming is built on four foundational principles. Understanding these deeply — not just memorising their definitions — is what separates a competent programmer from a software engineer who can design maintainable, scalable systems.

Pillar 1: Abstraction — Hiding Complexity

Abstraction means presenting a simplified interface to the user of a class, hiding the implementation details behind that interface. The user knows what a method does, not how it does it.

- **Example:** `employee.increase_salary(10)` — the caller doesn't need to know whether the salary is stored as annual or hourly, whether there is a cap on increases, whether a database is updated, or whether a log entry is written. All of that complexity is hidden inside the method.
- **Real-world analogy:** When you press a car's accelerator pedal, you don't need to understand fuel injection, combustion chemistry, or transmission mechanics. The pedal is the abstract interface.

- **Data Science relevance:** `model.fit(X_train, y_train)` — this one line hides hundreds of lines of gradient descent, matrix operations, and hyperparameter management.

Pillar 2: Encapsulation — Protecting Data

Encapsulation is the practice of bundling data (attributes) and the methods that operate on that data together inside a class, and restricting direct access to the raw data from outside the class. Objects expose a controlled interface (methods) rather than their internal state.

- **Why it matters:** Without encapsulation, any code anywhere in your program could directly modify `employee.salary = -50000`. With encapsulation, all salary changes go through `employee.increase_salary()` or `employee.set_salary()`, which can validate, log, and enforce business rules.
- **Python convention:** Prefix attributes with a single underscore (`_salary`) to signal 'private by convention'. Double underscore (`__salary`) triggers name mangling — stronger but not absolute protection.
- **Properties (@property):** Python's `@property` decorator allows you to expose attributes with controlled getter and setter access — you can add validation, logging, or computation behind a clean attribute-like interface.

Pillar 3: Inheritance — Reusing and Extending

Inheritance allows a new class (child class / subclass) to acquire all the attributes and methods of an existing class (parent class / superclass), and then extend or override them. This implements the **DRY principle** (Don't Repeat Yourself) — common functionality lives in the parent class; specialised functionality lives in children.

- **'is-a' relationship:** Inheritance should always represent an 'is-a' relationship. An Employee IS-A Person. A Developer IS-A Employee. A Labrador IS-A Dog. If you cannot say 'B is-a A', B should not inherit from A.
- **Method overriding:** A child class can redefine a method from its parent with specialised behaviour. `developer.increase_salary()` might apply a 15% bonus cap; `tester.increase_salary()` might apply a 10% cap.
- **super():** The `super()` function allows a child class to call the parent's version of a method. Used in `__init__` to initialise inherited attributes before adding child-specific ones.

The Employee Inheritance Hierarchy:

Person (parent): `name, age` → `greet()`, `get_age()`

Employee(Person) (child): `+ salary, position` → `increase_salary()`, `info()`

Developer(Employee) (grandchild): `+ framework` → `increase_salary()` [overridden with 15% cap], `write_code()`

Tester(Employee) (grandchild): `+ test_suite` → `increase_salary()` [overridden with 10% cap], `run_tests()`

Every Developer IS-A Employee IS-A Person. A Developer object can use methods from all three levels of the hierarchy.

Pillar 4: Polymorphism — One Interface, Many Behaviours

Polymorphism (Greek: 'many forms') means that different object types can respond to the same method call in their own, type-specific way. The calling code doesn't need to know which specific type it is dealing with — it calls the method, and the correct implementation runs automatically.

- **Example:** Both Developer and Tester have an `increase_salary()` method. The same line of code — `employee.increase_salary()` — does different things depending on whether employee is a Developer or a Tester.
- **Duck typing:** Python's polymorphism is largely achieved through duck typing: 'if it walks like a duck and quacks like a duck, it's a duck.' Python doesn't check the object's type; it just tries to call the method. If the method exists, it succeeds.
- **Benefits:** Polymorphism eliminates long if/elif/else chains that check object types. Instead of 'if type is Developer: do X, elif type is Tester: do Y', you just call `object.method()` and polymorphism routes to the correct implementation.

1.9 Class Attributes vs. Instance Attributes

Not all data in a class belongs to individual objects. Some data is shared across all objects of the same class. Python distinguishes these two levels clearly:

Attribute Type	Definition, Storage, and Use Case
Instance Attribute	Defined inside <code>__init__</code> using <code>self.attribute = value</code> . Each object has its own independent copy. Changes to one object's instance attributes do not affect any other object. Use for: data that varies per object (employee name, salary, age).
Class Attribute	Defined at the class level, outside any method. All objects share a single copy. Changing a class attribute changes it for all objects simultaneously (unless an object's instance attribute shadows it). Use for: constants, counters, shared configuration (<code>company_name = 'TechCorp'</code> , <code>employee_count = 0</code>).

A common pattern: a class attribute `employee_count` is incremented inside `__init__` every time a new employee is created, giving you a real-time count of all Employee instances without needing an external counter.

1.10 Class Introspection: Inspecting Objects at Runtime

Python provides powerful built-in tools for inspecting objects and classes at runtime — a capability called introspection. This is extensively used by frameworks, IDEs, debuggers, and serialisation libraries:

- **`type(obj)`:** Returns the class of an object. `type(employee1) → <class '__main__.Employee'>`
- **`isinstance(obj, cls)`:** Returns True if `obj` is an instance of `cls` or any subclass. `isinstance(developer1, Employee) → True`. Essential for type checking in polymorphic code.

- **issubclass(child, parent):** Returns True if child is a subclass of parent. `issubclass(Developer, Employee) → True`.
- **obj.__class__:** The class attribute on every object. Returns its class.
- **cls.__dict__:** A dictionary containing all attributes and methods defined in the class. `Employee.__dict__` shows `__init__`, `increase_salary`, `salary`, etc.
- **dir(obj):** Returns a sorted list of all attributes and methods available on an object, including inherited ones. Essential for debugging and exploration.

1.11 __slots__: Memory Optimisation for Large Object Counts

By default, Python stores each object's instance attributes in a dictionary (`__dict__`). Dictionaries are flexible but carry significant memory overhead — each object's `__dict__` consumes roughly 200–400 bytes of additional memory beyond the attribute values themselves.

For applications that create millions of objects (financial tick data, sensor readings, graph nodes), this overhead is significant. The `__slots__` declaration tells Python to pre-allocate a fixed set of attribute slots for each object, bypassing the per-object dictionary entirely:

- **Memory reduction:** Objects with `__slots__` typically use 40–60% less memory than equivalent objects with `__dict__`.
- **Speed improvement:** Attribute access is faster with `__slots__` (direct slot access vs. dictionary lookup).
- **Trade-off:** You can no longer add new attributes to objects at runtime — the slot list is fixed. Also, `__slots__` does not automatically inherit to subclasses.

1.12 Data Classes: Eliminating Boilerplate

A very common pattern in Python is creating classes that are primarily used to store structured data — essentially named containers for related values. Writing the full `__init__`, `__repr__`, and `__eq__` methods by hand for every such class is tedious and error-prone.

Python 3.7 introduced the `@dataclass` decorator from the `dataclasses` module. It automatically generates these methods based on class-level type annotations, dramatically reducing boilerplate while maintaining all OOP benefits:

Feature	What @dataclass Generates Automatically
<code>__init__</code>	A constructor that accepts all annotated fields as parameters
<code>__repr__</code>	A string representation: <code>Employee(name='Ji-Soo', age=38, salary=1200)</code>
<code>__eq__</code>	Equality comparison: two <code>Employee</code> objects are equal if all their fields are equal

<code>__lt__</code> , <code>__gt__</code> etc.	Ordering methods, if <code>order=True</code> is passed to <code>@dataclass</code>
<code>frozen=True</code>	Makes all fields immutable (like a named tuple) — no attribute can be changed after creation

1.13 Why OOP Matters for Data Science and GenAI Engineering

For aspiring Data Scientists and GenAI engineers, OOP is not abstract theory — it is the practical underpinning of every major tool and framework in the ecosystem:

Library / Framework	OOP Concept in Use	Practical Example
Pandas	Class with attributes + methods	<code>DataFrame</code> is a class. <code>df.describe()</code> , <code>df.groupby()</code> , <code>df.merge()</code> are methods.
Scikit-Learn	Inheritance + Polymorphism	All estimators inherit from <code>BaseEstimator</code> . <code>model.fit()</code> , <code>model.predict()</code> work the same regardless of whether model is <code>LinearRegression</code> or <code>RandomForest</code> .
PyTorch	Inheritance + Dunder Methods	Neural network layers are classes inheriting from <code>nn.Module</code> . The <code>forward()</code> method is overridden to define computation.
TensorFlow/Keras	Inheritance + Encapsulation	Layers, models, and callbacks are all classes. Internal gradient computation is encapsulated behind <code>model.compile()</code> and <code>model.fit()</code> .
Multi-Agent AI (MAS)	Composition + Inheritance	<code>BaseAgent</code> class (Chapter 5). Specialist agents inherit from it. Orchestrator composes them.
Data Pipelines	Encapsulation + Abstraction	Pipeline class encapsulates <code>load</code> → <code>validate</code> → <code>transform</code> → <code>save</code> steps. Caller just calls <code>pipeline.run()</code> .

SECTION 02 | Practical Implementation & Code

Section 2: Practical — Building Classes in Python, Step by Step

2.1 Setup

No external packages are required for core OOP. The examples in this chapter use only Python 3.7+ standard library. The dataclasses module is part of the standard library from Python 3.7 onwards.

Cell 1 — Your First Class: Employee Blueprint

```
# Cell 1: Defining a class and creating objects from it
# This is the foundation of all OOP in Python

class Employee:
    """
    Represents an employee in an organisation.

    Attributes:
        name      (str): Employee's full name.
        age       (int): Employee's age in years.
        position  (str): Job title / position.
        salary    (int): Monthly salary in dollars.
    """

    # — CLASS ATTRIBUTE —————
    # Shared across ALL Employee objects (not per-object)
    # Tracks the total number of employees created
    employee_count = 0
    company_name   = 'TechCorp Inc.'

    # — CONSTRUCTOR —————
    def __init__(self, name: str, age: int, position: str, salary: int):
        """
        Initialise a new Employee instance.
        Called automatically when Employee(...) is invoked.
```

```
'self' is the newly created object - Python passes it automatically.
"""

# — INSTANCE ATTRIBUTES — unique to each object —
self.name      = name      # Each employee has their own name
self.age       = age       # Each employee has their own age
self.position  = position  # Each employee has their own position
self.salary    = salary    # Each employee has their own salary


# Increment the shared class attribute every time a new employee is
created
Employee.employee_count += 1
print(f'[+] Created employee: {self.name} (Total:
{Employee.employee_count})')


# — INSTANCE METHODS —
def info(self):
    """Display information about this employee."""
    print(f'{self.name} is {self.age} years old.')
    print(f'Position: {self.position} | Salary: ${self.salary:,}')


def increase_salary(self, percent: float):
    """Increase this employee's salary by the given percentage."""
    if percent <= 0:
        raise ValueError('Salary increase must be positive.')
    increase = self.salary * (percent / 100)
    self.salary = int(self.salary + increase)
    print(f'{self.name}\''s salary increased by {percent}% to
    ${self.salary:,}')


# — DUNDER METHODS —
def __str__(self):
    """Human-readable string. Called by print() and str()."""
    return f'Employee({self.name}, {self.position}, ${self.salary:,})'


def __repr__(self):
    """Developer-readable string. Called by repr() and in the REPL."""
    return (f'Employee(name={self.name!r}, age={self.age}, '
            f'position={self.position!r}, salary={self.salary})')


def __eq__(self, other):
    """Two employees are equal if they have the same name and position."""
    if not isinstance(other, Employee):
        return NotImplemented
    return self.name == other.name and self.position == other.position
```

```

# — Instantiation: creating objects from the class —————
employee1 = Employee('Ji-Soo', 38, 'developer', 1200)
employee2 = Employee('Lauren', 44, 'tester', 1000)
employee3 = Employee('Marcus', 29, 'developer', 1100)

# — Instance attributes are independent —————
print('\n--- Instance Attribute Independence ---')
print(employee1.name)    # Ji-Soo
print(employee2.name)    # Lauren

employee1.name = 'Cynthia' # Only affects employee1
print(employee1.name)     # Cynthia
print(employee2.name)     # Lauren — unchanged

# — Class attribute is shared —————
print(f'\nTotal employees: {Employee.employee_count}') # 3
print(f'Company: {Employee.company_name}')

# — Dunder methods in action —————
print('\n--- Dunder Methods ---')
print(employee2)          # Calls __str__ → Employee(Lauren, tester, $1,000)
print(repr(employee2))    # Calls __repr__ → Employee(name='Lauren', ...)

# — Method calls —————
print('\n--- Method Calls ---')
employee2.info()
employee2.increase_salary(20)
employee2.info()

# — Introspection —————
print('\n--- Introspection ---')
print(type(employee1))    # <class '__main__.Employee'>
print(isinstance(employee1, Employee)) # True
print(employee2.__class__) # <class '__main__.Employee'>

```



Explanation — Cell 1:

class Employee: — The class keyword begins the class definition. By convention, class names use PascalCase (CapitalisedEachWord).

Docstring: The triple-quoted string immediately after class `Employee`: is the class docstring. It describes the class's purpose and attributes. Accessible via `Employee.__doc__` or `help(Employee)`.

`employee_count = 0`: A class attribute — defined outside any method, at the class level. Shared by all `Employee` objects. We access and modify it via `Employee.employee_count` (not `self.employee_count`) to make the scope explicit.

`__init__(self, name, age, position, salary)`: The constructor. The four parameters after `self` become instance attributes via `self.name = name` etc. This is the only place these attributes are defined — `__init__` guarantees every `Employee` object starts with all four attributes populated.

`self.salary = salary`: Creates an instance attribute called `salary` on this specific object. The right-hand `salary` is the parameter value passed by the caller; `self.salary` is the new attribute on the object.

`Employee.employee_count += 1`: We use `Employee.employee_count` (not `self.employee_count`) intentionally. If we wrote `self.employee_count += 1`, Python would create a new instance attribute on that specific object rather than updating the shared class attribute.

`info()` and `increase_salary()`: Instance methods — they take `self` as their first parameter. Python automatically passes the calling object as `self` when you write `employee2.increase_salary(20)`.

`__str__` vs `__repr__`: `__str__` is for humans (readable, friendly). `__repr__` is for developers (precise, should ideally be eval-able to recreate the object). Always implement both.

Cell 2 — Encapsulation with Properties

```
# Cell 2: Encapsulation – protecting data with properties
# Shows how to control access to sensitive attributes

class SecureEmployee:
    """
    Employee class with encapsulated salary.
    Salary can only be changed through validated setter methods.
    """

    def __init__(self, name: str, age: int, position: str, salary: int):
        self.name      = name
        self.age       = age
        self.position  = position
        # Single underscore: 'private by convention'
        # Outside code SHOULD NOT access _salary directly
        self._salary   = salary
        self._salary_history = [salary] # Audit trail

    @property
```

```
def salary(self):
    """
    Getter: allows reading salary as if it's a plain attribute.
    Called when you write: employee.salary (no parentheses)
    """
    return self._salary

@salary.setter
def salary(self, new_salary: int):
    """
    Setter: called when you write: employee.salary = 1500
    Validates the new value before applying it.
    """
    if not isinstance(new_salary, (int, float)):
        raise TypeError(f'Salary must be numeric, got {type(new_salary)}')
    if new_salary < 0:
        raise ValueError(f'Salary cannot be negative: {new_salary}')
    if new_salary > 1_000_000:
        raise ValueError(f'Salary exceeds maximum allowed: {new_salary}')
    self._salary_history.append(new_salary)
    self._salary = new_salary
    print(f'Salary updated to ${new_salary:,}')

def get_salary_history(self) -> list:
    """Return the full history of salary changes."""
    return self._salary_history.copy() # Return a copy, not the original

def increase_salary(self, percent: float):
    """Increase salary by percent. Routes through the validated setter."""
    new_salary = int(self._salary * (1 + percent / 100))
    self.salary = new_salary # Uses the @salary.setter - validates automatically

# — Usage —————
emp = SecureEmployee('Ji-Soo', 38, 'developer', 1200)

# Read salary via property (clean attribute-like syntax)
print(f'Current salary: ${emp.salary:,}')

# Modify salary via property setter (validates automatically)
emp.salary = 1400 # Calls the setter
emp.increase_salary(10) # Also routes through setter
```

```
# View audit trail
print(f'Salary history: {emp.get_salary_history()}')

# Try to set an invalid salary
try:
    emp.salary = -500 # Raises ValueError
except ValueError as e:
    print(f'Error caught: {e}')
```

Explanation — Cell 2:

@property: A decorator that transforms a method into a 'computed attribute'. Code that reads `emp.salary` looks exactly like accessing a plain attribute — no parentheses needed — but it secretly calls the getter method. This is the cleanest way to add logic (validation, logging, transformation) to attribute access without changing the external interface.

@salary.setter: Transforms a method into the setter for the salary property. When code writes `emp.salary = 1400`, Python silently calls this method with `new_salary=1400`. The external interface (`emp.salary = X`) looks identical to setting a plain attribute.

Single underscore `_salary`: The actual storage attribute. The leading underscore is Python's convention for 'private — hands off'. It's not enforced by the language, but it's a strong signal to other developers that this attribute is an internal implementation detail.

`self._salary_history.copy()`: We return a COPY of the history list, not the original. If we returned the original and the caller modified it (e.g., `history.append(999)`), they'd corrupt the internal state. Returning a copy protects encapsulation.

`increase_salary` routes through the setter: `self.salary = new_salary` (not `self._salary = new_salary`). This ensures that all salary changes — whether direct assignment or percentage increases — go through the same validation logic. This is the power of encapsulation: one place to enforce business rules.

Cell 3 — Inheritance and Method Overriding

```
# Cell 3: Inheritance — building a class hierarchy
# Person → Employee → Developer/Tester

class Person:
    """Base class representing a human being."""

    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def greet(self):
        """A greeting that works for any Person."""
```

```
        return f'Hi, my name is {self.name} and I am {self.age} years old.'

    def __str__(self):
        return f'Person({self.name}, age {self.age})'

class Employee(Person):    # Employee inherits from Person
    """
    An Employee IS-A Person with additional work-related attributes.
    Inherits all Person attributes and methods automatically.
    """

    def __init__(self, name: str, age: int, position: str, salary: int):
        # super().__init__() calls the PARENT's __init__
        # This initialises the inherited attributes (name, age)
        # without duplicating code
        super().__init__(name, age)

        # Now add Employee-specific attributes
        self.position = position
        self.salary = salary

    def info(self):
        """Display employee details."""
        print(f'{self.name} is {self.age} years old.')
        print(f'Role: {self.position}, Salary: ${self.salary:,}')

    def increase_salary(self, percent: float):
        """Increase salary - base implementation."""
        self.salary = int(self.salary * (1 + percent / 100))
        return self.salary

    def __str__(self):
        return f'Employee({self.name}, {self.position}, ${self.salary:,})'

class Developer(Employee):    # Developer inherits from Employee
    """
    A Developer IS-AN Employee who also knows a tech framework.
    Overrides increase_salary with a developer-specific cap.
    """

    MAX_INCREASE_PERCENT = 20    # Class constant
```



```
def __init__(self, name: str, age: int, salary: int, framework: str):
    # Initialise all Employee (and Person) attributes via super()
    super().__init__(name, age, 'developer', salary)
    # Add Developer-specific attribute
    self.framework = framework

def increase_salary(self, percent: float):
    """
    Override: Developers have a 20% cap on salary increases.
    This is POLYMORPHISM – same method name, different behaviour.
    """
    if percent > self.MAX_INCREASE_PERCENT:
        print(f'Cap applied: {percent}% reduced to
{self.MAX_INCREASE_PERCENT}%')
        percent = self.MAX_INCREASE_PERCENT
    return super().increase_salary(percent) # Call parent's
implementation

def write_code(self, task: str) -> str:
    """Developer-only method."""
    return f'{self.name} is writing {self.framework} code for: {task}'

def __str__(self):
    return f'Developer({self.name}, {self.framework}, ${self.salary:,})'

class Tester(Employee): # Tester inherits from Employee
    """A Tester IS-AN Employee who runs test suites."""

    MAX_INCREASE_PERCENT = 10 # Testers have a lower cap

    def __init__(self, name: str, age: int, salary: int, test_suite: str):
        super().__init__(name, age, 'tester', salary)
        self.test_suite = test_suite

    def increase_salary(self, percent: float):
        """Override: Testers capped at 10%."""
        if percent > self.MAX_INCREASE_PERCENT:
            print(f'Tester cap applied: {percent}% →
{self.MAX_INCREASE_PERCENT}%')
            percent = self.MAX_INCREASE_PERCENT
        return super().increase_salary(percent)
```

```

def run_tests(self) -> str:
    """Tester-only method."""
    return f'{self.name} is running test suite: {self.test_suite}'

# — Instantiation —————
dev1      = Developer('Ji-Soo', 38, 1200, 'Django')
tester1   = Tester('Lauren', 44, 1000, 'pytest')

# — Inherited methods work on child objects —————
print(dev1.greet())      # Inherited from Person — works on Developer!
dev1.info()              # Inherited from Employee

# — Polymorphism: same call, different behaviour —————
print('\n--- Polymorphism Demo ---')
employees = [dev1, tester1]
for emp in employees:
    print(f'{emp.name}: ', end='')
    emp.increase_salary(25) # Developer caps at 20%, Tester at 10%

# — Type checking —————
print('\n--- Type Checking ---')
print(isinstance(dev1, Developer)) # True
print(isinstance(dev1, Employee))  # True — Developer IS-AN Employee
print(isinstance(dev1, Person))     # True — Developer IS-A Person
print(isinstance(dev1, Tester))     # False — Developer is NOT a Tester

# — Method Resolution Order (MRO) —————
print('\nMethod Resolution Order:')
print([cls.__name__ for cls in Developer.__mro__])
# → ['Developer', 'Employee', 'Person', 'object']

```

Explanation — Cell 3:

`class Employee(Person):` — The parent class is specified in parentheses. Employee inherits ALL of Person's attributes and methods. This single line of code makes all of Person's functionality available in Employee without any copying.

`super().__init__(name, age):` Calls the parent class's `__init__`. This is essential — if you skip it, the inherited attributes (name, age) won't be initialised. `super()` returns a proxy object that delegates method calls to the parent class. Always call `super().__init__()` as the FIRST line of your child's `__init__`.

`Developer.increase_salary()` overrides `Employee.increase_salary()`: When Python resolves a method call, it follows the Method Resolution Order (MRO) — it looks in the object's class

first, then the parent, then the grandparent. Since Developer defines `increase_salary`, Python uses that version instead of Employee's.

`super().increase_salary(percent)` inside Developer: After applying the cap, the Developer's `increase_salary` calls the parent's implementation to actually perform the salary update. This avoids duplicating the salary calculation logic — inheritance + `super()` = maximum code reuse.

Polymorphism in the loop: `for emp in employees: emp.increase_salary(25)` — this code works identically regardless of whether `emp` is a Developer or a Tester. Python dispatches to the correct `increase_salary()` implementation based on the object's actual type. No `isinstance` checks needed.

`isinstance(dev1, Employee) → True`: Even though `dev1` was created from Developer, `isinstance` correctly reports that it IS-AN Employee (and IS-A Person). This is the power of inheritance — the object satisfies all levels of the hierarchy.

Cell 4 — `__slots__` and Memory Optimisation

```
# Cell 4: __slots__ - memory optimisation for large-scale object creation
# Demonstrates the performance difference with concrete measurements

import sys
import time

# — WITHOUT __slots__ (default Python objects) —————
class EmployeeNormal:
    """Standard class - stores attributes in a per-object __dict__."""
    def __init__(self, name: str, age: int, salary: int):
        self.name = name
        self.age = age
        self.salary = salary

# — WITH __slots__ —————
class EmployeeSlotted:
    """
    Class with __slots__ - pre-allocated attribute storage.
    No per-object __dict__: significantly lower memory footprint.
    """
    __slots__ = ['name', 'age', 'salary']

    def __init__(self, name: str, age: int, salary: int):
        self.name = name
        self.age = age
        self.salary = salary
```

```
# — Memory comparison —————
normal    = EmployeeNormal('Ji-Soo', 38, 1200)
slotted   = EmployeeSlotted('Ji-Soo', 38, 1200)

# sys.getsizeof gives the shallow size of the object itself
print(f'EmployeeNormal size: {sys.getsizeof(normal):>6} bytes')
print(f'EmployeeSlotted size: {sys.getsizeof(slotted):>6} bytes')

# For EmployeeNormal, add the __dict__ overhead
print(f'EmployeeNormal __dict__ overhead: {sys.getsizeof(normal.__dict__)} bytes')

try:
    _ = slotted.__dict__ # Should raise AttributeError
except AttributeError:
    print('EmployeeSlotted has NO __dict__ — memory saved!')

# — Creation speed comparison —————
N = 1_000_000 # 1 million objects

start = time.perf_counter()
normal_employees = [EmployeeNormal(f'Emp{i}', 30, 1000) for i in range(N)]
t_normal = time.perf_counter() - start

start = time.perf_counter()
slotted_employees = [EmployeeSlotted(f'Emp{i}', 30, 1000) for i in range(N)]
t_slotted = time.perf_counter() - start

print(f'\n1M objects creation:')
print(f' Normal   : {t_normal:.2f}s')
print(f' Slotted  : {t_slotted:.2f}s')
print(f' Speedup  : {t_normal/t_slotted:.2f}x')

# — __slots__ restriction: cannot add new attributes —————
try:
    slotted.department = 'Engineering' # Not in __slots__
except AttributeError as e:
    print(f'\n__slots__ restriction: {e}')
    print('This is the trade-off: rigidity in exchange for performance.')
```

 Explanation — Cell 4:

`__slots__ = ['name', 'age', 'salary']`: Tells Python to pre-allocate exactly three attribute slots per object instead of creating a per-object `__dict__`. Python creates C-level slot descriptors — faster and smaller than dictionary entries.

`sys.getsizeof()`: Returns the memory size in bytes of the object itself (shallow measurement). The key additional cost for normal objects is their `__dict__` — typically 200–400 bytes — which `__slots__` eliminates entirely.

When to use `__slots__`: When creating large numbers of objects (thousands to millions). Common scenarios in Data Science: graph nodes, sensor readings, financial tick data, feature vectors before they're loaded into a NumPy array. When NOT to use: general-purpose classes where you may need to add attributes dynamically.

The trade-off: You cannot add attributes beyond those declared in `__slots__`. `slotted.department = 'Engineering'` raises `AttributeError`. This rigidity is the price of performance.

Inheritance and `__slots__`: If a subclass does not declare its own `__slots__`, it will have a `__dict__` again (partially defeating the purpose). For full memory savings across a hierarchy, EVERY class in the chain must declare `__slots__`.

Cell 5 — Data Classes: Modern Python OOP

```
# Cell 5: @dataclass — eliminate boilerplate for data-centric classes
# Python 3.7+ — standard library, no installation required

from dataclasses import dataclass, field, asdict, astuple
from typing import List

# — Basic @dataclass —————
@dataclass
class Employee:
    """
    @dataclass automatically generates __init__, __repr__, and __eq__.
    Annotated fields (name: str) become __init__ parameters in order.
    """
    name      : str
    age       : int
    salary    : int
    position: str = 'developer'    # Default value: position is optional

    # Custom method — works exactly like in a regular class
    def increase_salary(self, percent: float):
        self.salary = int(self.salary * (1 + percent / 100))
        return self.salary
```

```
# — @dataclass with advanced options —————
@dataclass(order=True, frozen=False)
class MLEExperiment:
    """
    Tracks a machine learning experiment run.
    order=True → generates __lt__, __gt__, __le__, __ge__ (enables sorting)
    frozen=False → attributes can be modified after creation (default)
    """
    # sort_index is used as the primary sort key (Python sorts on first field
    by default)
    accuracy      : float
    model_name     : str
    learning_rate  : float
    epochs        : int
    # field(default_factory=list) creates a NEW empty list for each instance
    # Never use: metrics: list = [] ← this shares one list across ALL
    instances!
    metrics        : List[float] = field(default_factory=list)

    def log_metric(self, metric: float):
        self.metrics.append(metric)

@dataclass(frozen=True) # Immutable — like a named tuple but with type hints
class ModelConfig:
    """
    frozen=True: All fields are read-only after creation.
    __hash__ is generated automatically (since the object is immutable).
    Frozen dataclasses can be used as dict keys or set members.
    """
    model_type     : str
    learning_rate  : float
    batch_size     : int
    dropout_rate   : float = 0.1

# — Usage —————
print('=== Basic Employee Dataclass ===')
emp1 = Employee('Ji-Soo', 38, 1200)
emp2 = Employee('Lauren', 44, 1000, 'tester')
emp3 = Employee('Ji-Soo', 38, 1200) # Same data as emp1

print(emp1) # __repr__ auto-generated
```

```

print(emp1 == emp3)          # True - __eq__ compares all fields
print(emp1 == emp2)          # False

emp1.increase_salary(10)
print(f'After raise: {emp1}')

print('\n=== ML Experiment Dataclass ===')
exp1 = MLExperiment(accuracy=0.94, model_name='RandomForest',
learning_rate=0.01, epochs=100)
exp2 = MLExperiment(accuracy=0.97, model_name='XGBoost',
learning_rate=0.05, epochs=200)
exp3 = MLExperiment(accuracy=0.91, model_name='LogisticReg',
learning_rate=0.1, epochs=50)

# Sort experiments by accuracy (order=True enables comparison operators)
experiments = [exp1, exp2, exp3]
experiments.sort(reverse=True)  # Highest accuracy first
print('Experiments ranked by accuracy:')
for i, exp in enumerate(experiments, 1):
    print(f' {i}. {exp.model_name}: {exp.accuracy:.2%}')

print('\n=== Frozen ModelConfig ===')
config = ModelConfig('TransformerEncoder', 1e-4, 32)
print(config)
print(f'Config hash: {hash(config)}')

# Attempting to modify a frozen dataclass raises FrozenInstanceError
try:
    config.learning_rate = 1e-3
except Exception as e:
    print(f'Cannot modify frozen dataclass: {type(e).__name__}')

# — Utility functions: asdict and astuple —————
print('\n=== Serialisation Utilities ===')
emp_dict = asdict(emp2)  # Convert to dict (great for JSON serialisation)
emp_tuple = astuple(emp2)  # Convert to tuple
print(f'As dict : {emp_dict}')
print(f'As tuple: {emp_tuple}')

```

Explanation — Cell 5:

@dataclass: The decorator that triggers auto-generation of `__init__`, `__repr__`, and `__eq__` based on the annotated class fields. It reads the type annotations and builds the methods at class-definition time — zero runtime overhead.

Field order matters: The generated `__init__` creates parameters in the same order as the field declarations. Fields with default values must come AFTER fields without defaults (just like regular function parameters).

`field(default_factory=list)`: CRITICAL — never write `metrics: list = []` in a dataclass (or in any class). A mutable default value is shared across ALL instances — modifying one instance's metrics modifies every instance's. `field(default_factory=list)` creates a fresh list for each instance via a factory function.

`order=True`: Generates `__lt__`, `__le__`, `__gt__`, `__ge__` by comparing objects as tuples of their fields. This makes dataclass instances sortable with `sorted()` and `sort()`. Comparison starts with the first field — put your primary sort key first, or use `sort(key=lambda x: x.accuracy)` for explicit control.

`frozen=True`: Makes all fields read-only after creation. Python generates `__hash__` automatically for frozen dataclasses — making them usable as dictionary keys or set members. Regular (non-frozen) dataclasses are unhashable by default because they're mutable.

`asdict()` and `astuple()`: Utility functions from the `dataclasses` module that convert a dataclass instance to a plain dict or tuple. `asdict()` is especially valuable for JSON serialisation (`json.dumps(asdict(emp))`) without needing to write custom serialisation code.

Cell 6 — OOP in Practice: A Data Science Pipeline Class

```
# Cell 6: OOP applied to Data Science – a reusable ML Pipeline class
# Demonstrates all four OOP pillars in a realistic DS context

from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
import json

@dataclass
class PipelineStep:
    """Represents a single step in a data processing pipeline."""
    name      : str
    status    : str = 'pending'  # pending | running | success | failed
    result    : Any = None
    error     : Optional[str] = None

class DataPipeline:
    """
    A reusable, configurable data processing pipeline.
    Demonstrates:
    - Encapsulation : internal steps list is managed through methods
    - Abstraction   : users call pipeline.run() without knowing the internals
    - Data Class    : PipelineStep is a @dataclass
    """
```



```
- OOP design      : state tracked, errors handled, results accessible
"""

def __init__(self, name: str, verbose: bool = True):
    self.name      = name
    self.verbose   = verbose
    self._steps    : List[PipelineStep] = []
    self._data     : Any = None
    self._status   : str = 'idle'
    self._log      : List[str] = []

def _log_message(self, msg: str):
    """Internal logging - encapsulated, not directly accessible."""
    entry = f'[{self.name}] {msg}'
    self._log.append(entry)
    if self.verbose:
        print(entry)

def load(self, data_source: str) -> 'DataPipeline':
    """
    Load data from source.
    Returns self for method chaining: pipeline.load().validate().run()
    """
    step = PipelineStep('load')
    step.status = 'running'
    self._log_message(f'Loading data from {data_source}...')
    try:
        # Simulate data loading
        self._data = {'source': data_source, 'records': 1000, 'features':
12}

        step.status = 'success'
        step.result = f'{self._data["records"]} records loaded'
    except Exception as e:
        step.status = 'failed'
        step.error = str(e)
    self._steps.append(step)
    return self # Enable method chaining

def validate(self) -> 'DataPipeline':
    """Validate loaded data against schema requirements."""
    step = PipelineStep('validate')
    step.status = 'running'
    self._log_message('Validating data schema...')
    if self._data is None:
```

```
        step.status = 'failed'
        step.error = 'No data loaded. Call load() first.'
    else:
        step.status = 'success'
        step.result = 'Schema valid'
    self._steps.append(step)
    return self

def transform(self, operations: List[str]) -> 'DataPipeline':
    """Apply transformation operations to the data."""
    step = PipelineStep('transform')
    step.status = 'running'
    self._log_message(f'Applying transformations: {operations}')
    step.status = 'success'
    step.result = f'Applied {len(operations)} transformations'
    self._steps.append(step)
    return self

def run(self) -> Dict[str, Any]:
    """
    Execute the pipeline and return a status report.
    ABSTRACTION in action: caller just calls .run() -
    all step management, error handling, and logging is hidden.
    """
    self._status = 'running'
    self._log_message('Pipeline execution complete.')

    failed_steps = [s for s in self._steps if s.status == 'failed']
    self._status = 'failed' if failed_steps else 'success'

    return {
        'pipeline' : self.name,
        'status'    : self._status,
        'steps'     : [{
            'name'   : s.name,
            'status' : s.status,
            'result' : s.result,
            'error'  : s.error
        } for s in self._steps],
        'errors'    : [s.error for s in failed_steps]
    }

def get_log(self) -> List[str]:
    """Access the pipeline execution log."""
```

```

        return self._log.copy()

    def __str__(self):
        step_summary = ', '.join(f'{s.name}:{s.status}' for s in self._steps)
        return f'DataPipeline({self.name}, status={self._status}, steps=[{step_summary}])'

    def __repr__(self):
        return f'DataPipeline(name={self.name!r}, verbose={self.verbose})'

# — Usage: method chaining pipeline —————
pipeline = DataPipeline('CustomerChurnPipeline', verbose=True)

report = (
    pipeline
    .load('s3://data-bucket/customer_data.csv')
    .validate()
    .transform(['normalise_salary', 'encode_categories', 'drop_nulls'])
    .run()
)

print('\n=== Pipeline Report ===')
print(json.dumps(report, indent=2))
print('\n', pipeline)

```

Explanation — Cell 6:

Return self (method chaining): Each method (load, validate, transform) returns self — the object itself. This enables the elegant fluent API: `pipeline.load(...).validate().transform(...).run()`. The caller writes code that reads like a sentence describing what the pipeline does.

PipelineStep @dataclass: Using a dataclass for each step gives us a structured, self-describing container for step metadata. Instead of passing around raw dicts (where keys could be misspelled), we have a typed object with documented fields.

self._steps (encapsulation): The steps list is prefixed with `_` — it's an internal implementation detail. External code doesn't manipulate steps directly; it calls `load()`, `validate()`, and `transform()` which manage the steps internally.

run() as abstraction: The caller doesn't know or care about step management, logging, or error aggregation. They just call `pipeline.run()` and receive a clean status report. The entire complexity of pipeline execution is abstracted behind this one method call.

OOP for Data Science — why this pattern matters: In production ML systems, you'll encounter `sklearn.Pipeline`, `MLflow` tracking, and `Apache Airflow DAGs` — all of which are

sophisticated versions of this exact pattern. Understanding how to design a DataPipeline class teaches you the mental model behind all of them.

2.2 Quick Reference: OOP Patterns at a Glance

Pattern / Feature	When to Use
@property + setter	When an attribute needs validation or logging on read/write. External interface stays clean (obj.attr = X).
super().__init__(...)	Always call in child class <code>__init__</code> before adding child attributes. Ensures inherited attributes are properly initialised.
<code>__str__</code> vs <code>__repr__</code>	Always implement both. <code>__str__</code> for end users; <code>__repr__</code> for developers and debugging.
Class attribute	Shared counters, constants, configuration that doesn't vary per object. Access via <code>ClassName.attr</code> , not <code>self.attr</code> .
<code>__slots__</code>	Thousands to millions of objects with fixed attribute sets. Memory and speed critical. Trade-off: no dynamic attribute addition.
@dataclass	Data-centric classes where the primary purpose is storing fields. Eliminates <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code> boilerplate.
@dataclass(frozen=True)	Immutable records, configuration objects, dictionary keys. Automatically hashable.
isinstance() check	Use instead of type() for polymorphic code. <code>isinstance(x, Employee)</code> returns True for Developer and Tester too.
Method chaining	Return self from methods to enable fluent API: <code>obj.method1().method2().method3()</code>

Key Takeaways

- A class is a blueprint; an object is a real instance. Classes define what attributes and methods every object will have. Each object gets its own private copy of instance attributes, completely independent from all other objects of the same class.
- The four OOP pillars — Abstraction, Encapsulation, Inheritance, Polymorphism — are not just academic concepts. They directly solve real engineering problems: abstraction reduces complexity; encapsulation protects data integrity; inheritance eliminates code duplication; polymorphism enables flexible, extensible designs without if/elif type-checking chains.
- The self parameter is fundamental to how Python OOP works. Python implicitly passes the calling object as the first argument to every instance method. self is how methods know which object's data to operate on.

- Dunder methods (`__str__`, `__repr__`, `__eq__`, `__len__`, `__add__`) are Python's hook system for customising how your objects interact with built-in operations. Always implement at minimum `__str__` and `__repr__` in every class you create.
- `super()` is the correct way to call parent class methods from a child class. Always call `super().__init__()` as the first line of a child's `__init__` to ensure inherited attributes are properly initialised.
- Use `@property` + setters for attribute validation and encapsulation without changing the external interface. `self._salary` with a `@property` getter gives callers the clean `emp.salary` syntax while protecting the data behind validation logic.
- `__slots__` is a targeted optimisation for memory-critical applications with large numbers of objects. It eliminates the per-object `__dict__` overhead at the cost of dynamic attribute flexibility.
- `@dataclass` eliminates boilerplate for data-centric classes. NEVER use a mutable default value (list, dict) directly — always use `field(default_factory=list)` to ensure each instance gets its own fresh container.
- OOP is the foundation of the entire Python Data Science ecosystem. Pandas DataFrame, Scikit-Learn estimators, PyTorch `nn.Module`, and TensorFlow Keras layers are all built on the exact principles covered in this chapter. Mastering OOP is mastering the language of production AI engineering.

Coming Up Next

Functional Programming in Python | Decorators & Context Managers | Design Patterns for Data Science